

Graph

**** No need to write the Edge class, addEdge() etc. functions unless specified in the problem****

****Just design the function(s) specified in the problem.****

****No need to write driver code.****

****This document is a compilation of previous semester's exam (midterm/final) problems and lab quiz problems. Try to practice relevant problems from online judges as well.****

1.

Tasks from Lab 9 - [w](#) Lab 9-GRAPH.docx

2.

Suppose you have a set of 10 vertices which are denoted as A, B, C, D, E, F, G, H, I, J.
The matrix representation below shows the relationship between any of these two vertices.

	A (0)	B (1)	C (2)	D (3)	E (4)	F (5)	G (6)	H (7)	I (8)	J (9)
A (0)		5	3							
B (1)				7						
C (2)				4	6					
D (3)					2					
E (4)										
F (5)							8			
G (6)								9		
H (7)									10	
I (8)										
J (9)										11

[Note: Vertex A is mapped as index position 0, Vertex B is mapped as index position 1 in the matrix and so on]

- Using the above matrix representation, draw an adjacency list representation of the above nodes. [3]
- Determine** whether the graph is directed or undirected. If it is undirected, transform it into a directed graph and if it is directed, transform it into an undirected one using the above matrix representation. Write down the resulting matrix in your script [1+2]
- Suppose that the vertices A to J have the following weights 31, 19, 25, 40, 45, 35, 33, 27, 14, 21, 37 respectively.
 - Draw a Binary Search Tree using the vertices A to J sequentially based on their weights. Write down the node's weights while drawing the BST. [2]
 - Delete the node with the key of value 31 with the help of its predecessor of the above tree and draw the new tree. [2]

3.

You are organizing a Networking Conference where participants can interact multiple times, forming an undirected network graph where:

- **Vertices** represent participants.
- **Edges** represent interactions (multiple edges indicate repeated interactions). 2 vertex having multiple edges between them means 2 participants interact multiple times.

A network graph is **complete** if every participant interacts with every other participant at least once.

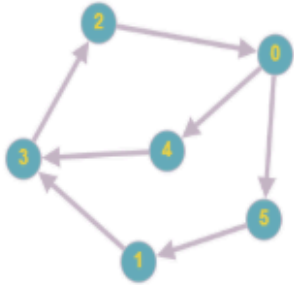
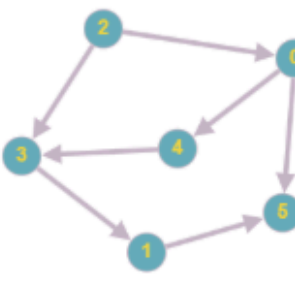
Write a function `is_complete` to determine if a given network graph, represented as an unweighted adjacency list, is complete or not. If complete the function will return `True`, else `False`. The **Edge** class has **source** (**Type: String**), **destination** (**Type: String**), and a **next** (**Type: Edge**) variable. You can use extra arrays if you want but cannot modify the given adjacency list.

Python Notation	Java Notation
<pre>def is_complete(network): #To do</pre>	<pre>static boolean is_complete(Edge[] network){ //To do }</pre>

Input	Output	Explanation
<pre>network = [0: (0, 1) → (0, 2) → (0, 3), 1: (1, 0) → (1, 2) → (1, 3), 2: (2, 0) → (2, 1) → (2, 3), 3: (3, 0) → (3, 1) → (3, 2)]]</pre>	True	There are 4 participants: 0, 1, 2, and 3. Every participant has met every other participant at least once: • Participant 0 met with participants 1, 2, and 3. • Participant 1 met with participants 0, 2, and 3. • Participant 2 met with participants 0, 1, and 3. • Participant 3 met with participants 0, 1, and 2. Since every pair of participants has met at least once, the network is complete.
<pre>network = [0: (0, 1) → (0, 2) → (0, 3), 1: (1, 0) → (1, 2) → (1, 0), 2: (2, 0) → (2, 1) → (2, 3), 3: (3, 1) → (3, 2) → (3, 2)]]</pre>	False	There are 4 participants: 0, 1, 2, and 3. • Participant 1 has not met with participant 3. • Participant 3 has not met with participant 0. Since there are missing interactions, the network is incomplete.

4.

You need to write a function called **reverseEdge()**. That takes a **directed graph** represented as an adjacency matrix in its parameter. Your task is to reverse all the outgoing connections from the **odd** vertices in the graph and return the matrix.

Sample Given Adjacency Matrix		Sample Output Adjacency Matrix	
	<pre>0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0,</pre>		<pre>0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0,</pre>
Sample Function Call			
<code>reverseEdge(graph)</code>			

Explanation: Each row in the adjacency matrix represents a Vertex, and the columns represent other Vertices where the outgoing connection is made. For example, in the sample input, the 1th row (0 0 0 1 0 0) means there is an outgoing edge from Vertex 1 to Vertex 3. After reversing the connection the outgoing edge would be going from Vertex 3 to Vertex 1. This is seen in the Row 3 (0 1 0 0 0 0) of the sample output. Moreover, the 3th row (0 0 1 0 0 0) means there is an outgoing edge from Vertex 3 to Vertex 2. After reversing the connection the outgoing edge would be going from Vertex 2 to Vertex 3. This is seen in the Row 2 (1 0 0 1 0 0) of the sample output. This reversing is only applicable for the **odd** vertices like 1,3,5.

5.

You need to write a function called **reverseEdge()**. That takes a **directed graph** represented as an adjacency matrix in its parameter. Your task is to reverse all the outgoing connections from the **even** vertices in the graph and then return the matrix.

Sample Given Adjacency Matrix		Sample Output Adjacency Matrix	
	<pre>0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0,</pre>		<pre>0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0,</pre>
Sample Function Call			
reverseEdge(graph)			

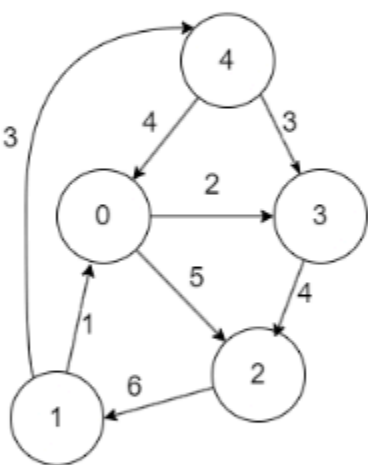
Explanation: Each row in the adjacency matrix represents a Vertex, and the columns represent other Vertices where the outgoing connection is made. For example, in the sample input, the 0th row (0 0 0 0 1 1) means there is an outgoing edge from Vertex 0 to Vertex 4 and Vertex 5. After reversing the connection the outgoing edge would be going from Vertex 4 to Vertex 0 and Vertex 5 to Vertex 0. This is seen in the Row 4 (1 0 0 0 0 0) and Row 5 (1 1 0 0 0 0) of the sample output. This reversing is only applicable for the **even** vertices like 0,2,4.

6.

Given a directed weighted graph, a starting vertex s and total available points p , traverse the graph, always choosing the outgoing edge with the **maximum weight at each step**. When traversing an edge with weight w , expend w points from total available points.

Determine the last vertex you can reach from s before running out of points. If you reach **dead end (no outgoing edges)** before using all your points, that vertex is the last reachable vertex.

Note: You can solve using either adjacency matrix or adjacency list. Assume that the **Graph** already created.

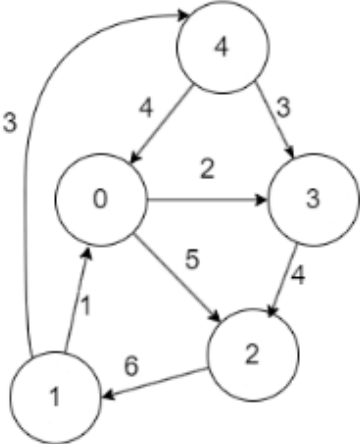
Given Graph & Sample Input	Sample Output
graph =  $s = 0$ $p = 15$ <code>findMinPath(graph, s, p)</code> Here, the "graph" parameter is a square Matrix if you're using the Adjacency Matrix. Otherwise, it is an array of singly Nodes if you're using Adjacency List.	Last reachable vertex = 4 Explanation: Choosing max weight edge from the starting point $s = 0$, i) vertex 0 $\rightarrow$ vertex 2 cost = 5, points available = $15 - 5 = 10$ ii) vertex 2 $\rightarrow$ vertex 1 cost = 6, points available = $10 - 6 = 4$ iii) vertex 1 $\rightarrow$ vertex 4 cost = 3, points available = $4 - 3 = 1$ Cannot go any further with available points. Therefore, last reachable vertex = 4

7.

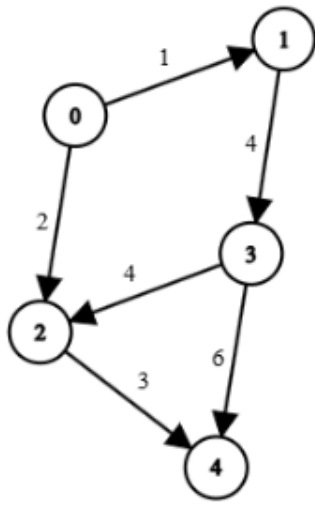
Given a directed weighted graph, a starting vertex s and total available points p , traverse the graph, always choosing the outgoing edge with the **minimum weight at each step**. When traversing an edge with weight w , expend w points from total available points.

Determine the last vertex you can reach from s before running out of points. If you reach a **dead end (no outgoing edges)** before using all your points, that vertex is the last reachable vertex.

Note: You can solve using either adjacency matrix or adjacency list. Assume that the **Graph** is already created.

Given Graph & Sample Input	Sample Output
graph =  $s = 0$ $p = 13$ <code>findMinPath(graph, s, p)</code> Here, the "graph" parameter is a square Matrix if you're using the Adjacency Matrix. Otherwise, it is an array of singly Nodes if you're using Adjacency List.	Last reachable vertex = 0 Explanation: Choosing min weight edge from the starting point $s = 0$, i) vertex 0 $\rightarrow$ vertex 3 cost = 2, points available = $13 - 2 = 11$ ii) vertex 3 $\rightarrow$ vertex 2 cost = 4, points available = $11 - 4 = 7$ iii) vertex 2 $\rightarrow$ vertex 1 cost = 6, points available = $7 - 6 = 1$ iv) vertex 1 $\rightarrow$ vertex 0 cost = 1, points available = $1 - 1 = 0$ Cannot go any further with available points. Therefore, last reachable vertex = 0

8.

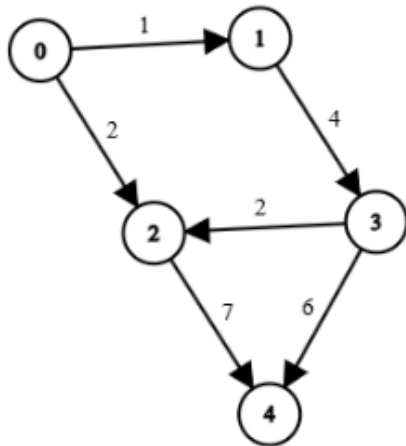


Sample Input	5, graph	Number of Vertices, The Graph
Sample Output	3,	Answer Vertex Index. This Vertex 3 has the most Outgoing edges ; for the same reason , Vertex 0 can also be the answer .
	[0,6]	Array of Outgoing edges. Here, outgoing edge 4!>5 which is why the Array has 0 in that place and outgoing edge 6>5 so, that weight is added. As vertex 0 can also be the answer the resulting array for that would be [0,0] .

For this **directed, edge-weighted** graph implemented using an **adjacency list**, find the vertex with the most **outgoing edges**. The vertex must have **at least 2** outgoing edges; otherwise, **return -1** (for Java an Empty array). After finding the vertex, **return** an array of all the **outgoing edges** from that vertex that have a weight **greater than 5**.

[Assume **Node** and **Graph** classes have been **implemented** but, you **cannot** use **Adjacency Matrix**. Here, the graph]

9.



Sample Input	5, graph	Number of Vertices, The Graph
Sample Output	1,	Answer Vertex Index. This Vertex 1 has the lowest Outgoing edges , and for the same reason, Vertex 2 can also be the answer .
	[4]	Array of Outgoing edges. Here, outgoing edge 4<5 which is why the Array has 4 weight. As vertex 2 can also be the answer the resulting array for that case would be [0] because 7!<5 so the weight cannot be added to the array.

For this **directed, edge-weighted** graph implemented using an **adjacency list**, find the vertex with the least amount of **outgoing edges**. The vertex must have **at least 1** outgoing edge; otherwise, **return -1** (for Java an Empty array). After finding the vertex, **return** an array of all the **outgoing edges** from that vertex that weigh **less than 5**.

[Assume **Node** and **Graph** classes have been **implemented** but, you **cannot** use **Adjacency Matrix**]

10.

Q1. [10 marks] You are given a directed, weighted graph represented using an adjacency list. Your task is to find the **difference** between **the sum of the N largest edge weights** and **the sum of the N smallest edge weights** in the graph.

Write a function `difference_Sum(Edge[] adjList, int N)`, that takes a directed, weighted graph (in adjacency list representation) and an integer N (where $1 \leq N \leq$ total number of edges) as input parameters, and returns the required difference. **You can't use any data structures other than heap to solve this problem.**

Keep in mind, the adjacency list is given as an array of **Edge** objects and the **Edge** class has the following fields: **source (Type: String)**, **destination (Type: String)**, **weight (Type: Integer)**, **next (Type: Edge)**. Assume **MaxHeap** or **MinHeap** classes with the **insert(item)** and **extract()** (extract will return the extracted value) methods are already implemented for you.

Input	Output	Explanation
0 → (0, 1, 4) → (0, 2, 7) 1 → (1, 3, 3) → (1, 4, 6) 2 → (2, 3, 2) 3 → (3, 0, 1) → (3, 4, 5) 4 → () (no outgoing edges)	12	Edge Weights: {4, 7, 3, 6, 2, 1, 5} For N=3: • 3 largest weights = {7, 6, 5} → sum = 18 • 3 smallest weights = {1, 2, 3} → sum = 6 • Difference = 18 - 6 = 12

Q2. [2 Marks] Draw the Example graph of **Q1** and write the adjacency matrix.

Q3. [2 Marks] Convert the adjacency matrix of the above directed graph of **Q2** into an undirected graph without deleting any existing edges.

Q4. [3 Marks] Consider the array [8, 2, 5, 1, 9, 7, 8, 3]. Write down the modified array when it will be converted to a Minheap. Again show the final array once **extractMin()** has been called on the previous modified array.

11.

2) You are given the adjacency list of an **undirected**, unweighted graph. If you add up the lengths of all the linked lists in the adjacency list, the total comes out to **20**. How many edges are there in the graph? What if the graph is directed?

3) Write one advantage of **Adjacency List** over **Adjacency Matrix**.

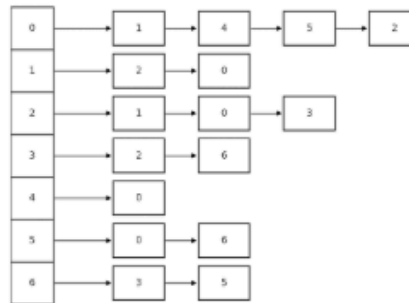
12.

You are given a social network represented as a graph. Given an **undirected-unweighted** friendship graph, where each **vertex** represents a **person** (labeled 0 to 6) and each **edge** represents a friendship between two people. [10 marks]

Write a function **areMutualFriends** that takes a graph represented as an adjacency list and determines whether two given persons **person1** and **person2** are mutual friends. Two persons are mutual friends if:

1. They **share at least one common friend**.
2. They are **not directly friends with each other**.

The function should return **True** (or **true**) if they are mutual friends, otherwise **False** (or **false**). Assume you are given a graph represented as an Adjacency List:



Sample Input	Sample Output
<pre> graph LR 1 --- Friend 0 2 --- Friend 0 0 --- Friend 4 0 --- Friend 5 2 --- Friend 3 3 --- Friend 6 4 --- Friend 5 5 --- Friend 6 </pre> 1. (person1 = 1, person2 = 4) 2. (person1 = 1, person2 = 6)	<pre> print(areMutualFriends(adjList, 1, 4)) # True print(areMutualFriends(adjList, 1, 6)) # False System.out.println(areMutualFriends(adjList, 1, 4)); // True System.out.println(areMutualFriends(adjList, 1, 6)); // False </pre> Explanation: In the first case (1, 4), Person 1 has neighbors {0, 2} and Person 4 has neighbors {0}. They share a common neighbor (0) but are not directly connected, so the result is True. In the second case (1, 6), Person 1 has neighbors {0, 2} and Person 6 has neighbors {3, 5}. They have no common neighbor, so the result is False.

13.

3. If an undirected weighted graph has 9 vertices, then what is the maximum number of undirected edges the graph can have? (**No self-loop**) [1 point]

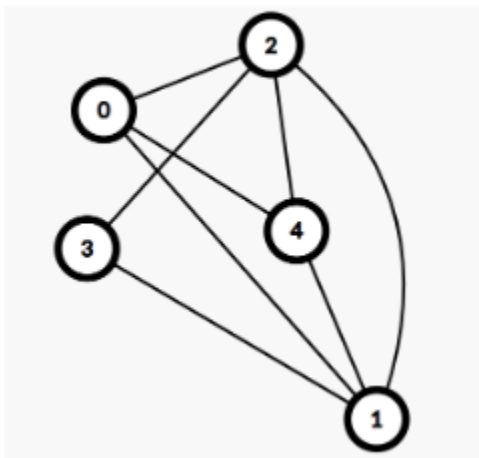
Answer:

4. For a directed unweighted graph with V vertices and E edges, what is the space complexity of storing it using an adjacency List representation? [1 point]

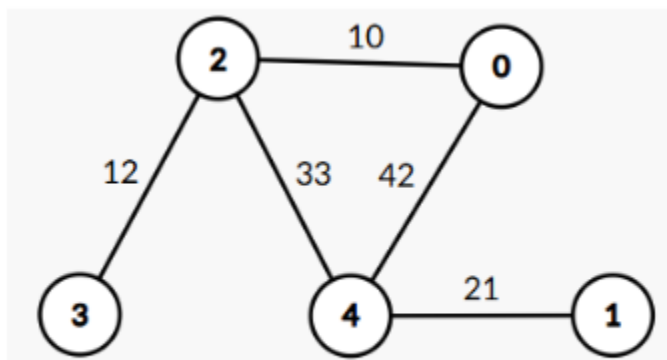
Answer:

14.

Represent the following undirected graph using an adjacency list:



15.



What will be the adjacency matrix of the graph shown above?

16.

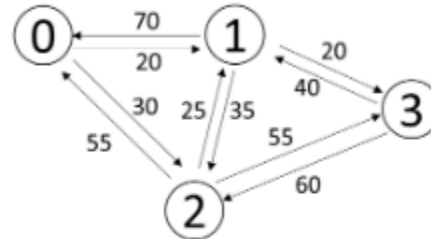
Q1. Which operation in a heap requires heapify? a) Searching for a key b) Deleting the root c) Traversing the heap d) Printing the heap elements	Q4. What is the shortest path length from vertex 0 to vertex 2? a) 1 b) 2 c) 3 d) 4																
Answer question Q2, Q3, Q4 using the following adjacency matrix representing a graph <table border="1"><tr><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>0</td></tr></table>	0	1	0	0	0	0	1	1	0	1	0	0	0	0	1	0	Q5. In an undirected graph, the degree of a vertex is: a) Number of outgoing edges only b) Number of incoming edges only c) Total number of edges incident on the vertex d) Always equal to the number of vertices
0	1	0	0														
0	0	1	1														
0	1	0	0														
0	0	1	0														
Q2. Is there any cycle in the above mentioned graph? a) Yes b) No	Q6. Which data structure is used in BFS graph traversal? a) Array b) Stack c) Queue d) Linked-List																
Q3. What's the out degree of Vertex 2 in the above mentioned graph? a) 1 b) 2 c) 3 d) 4	Q7. What's the time complexity of Swim operation in heap? a) $O(n)$ b) $O(n \cdot \log(n))$ c) $O(n^2)$ d) $O(\log(n))$																

17.

Question – 5: CO2 [8 Points]

The National Transport Authority monitors traffic volumes between cities every day. They are using a network modeled as a directed, weighted graph, where:

- Vertices represent cities
- Directed edges represent one-way roads
- Edge weights represent the average number of vehicles per day traveling from the source city to the destination city



The authority now wants to identify the city that has the least total incoming traffic, which refers to the vertex with the minimum sum of incoming edge weights. For the given example in the figure, the output should be city 1.

Now write a method/function that takes a directed, weighted graph (adjacency list) as an input parameter and returns the city with the minimum total incoming traffic volume. You can assume that the Edge class has source (Type: String), destination (Type: String), weight (Type: Integer), and a next (Type: Edge) variable. Note that the adjacency list represents the out-degree of the vertices.

Python Notation	Java Notation
<pre>def find_minimum(graph): #write your code here</pre>	<pre>static String find_minimum(Edge [] graph){ //write your code here }</pre>

Sample Input	Output	Explanation																
<table><tr><td>0</td><td>1</td><td>2</td><td>3</td></tr><tr><td><0,1,20></td><td><1,0,70></td><td><2,0,55></td><td><3,1,40></td></tr><tr><td><0,2,30></td><td><1,2,35></td><td><2,1,25></td><td><3,2,60></td></tr><tr><td></td><td><1,3,20></td><td><2,3,55></td><td></td></tr></table>	0	1	2	3	<0,1,20>	<1,0,70>	<2,0,55>	<3,1,40>	<0,2,30>	<1,2,35>	<2,1,25>	<3,2,60>		<1,3,20>	<2,3,55>		1	The sum of the incoming edge weights for vertex 1 is: 20+25+40 = 85 which is the minimum among all. Hence, 1 is returned.
0	1	2	3															
<0,1,20>	<1,0,70>	<2,0,55>	<3,1,40>															
<0,2,30>	<1,2,35>	<2,1,25>	<3,2,60>															
	<1,3,20>	<2,3,55>																

18.

Question – 5: CO2 [4 Points]

	0	1	2	3	4	5	6	7
0	0	6	0	0	7	0	0	0
1	6	0	4	0	8	0	0	3
2	0	4	0	2	0	1	8	0
3	0	0	2	0	9	0	0	0
4	7	8	0	9	0	0	0	0
5	0	0	1	0	0	0	9	0
6	0	0	8	0	0	9	0	0
7	0	3	0	0	0	0	0	0

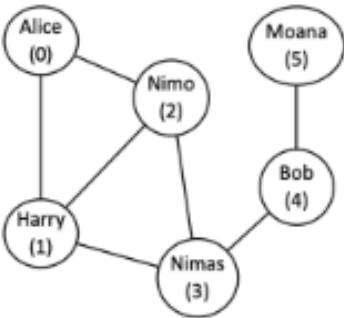
The adjacency matrix shown above represents a weighted graph

- Draw the Graph. [2 marks]
- Show the traversal sequence by performing a special Depth First Search (DFS) traversal of the graph starting from vertex 2. The DFS is special because when a vertex has multiple unvisited neighbors, the traversal must visit the neighbors in ascending order of their edge weights, meaning the one with the smallest weight will be chosen to visit first. [2 mark]

Note: You do not need to write any code for this question, just show the simulation.

19.

You are given an **Undirected Unweighted Graph** representing a Facebook friendship network, where each vertex is a person and each edge is a friendship. Two people are said to have a mutual friend if there exists a third person who is directly connected to both of them. For example, Harry and Bob have one mutual friend, Nimas.



Your task is to write a function that finds the pair of distinct people that has the maximum number of mutual friends. If multiple pairs tie for the maximum, print any one of them. If no valid pair exists with at least one mutual friend, print the message, **“There are no mutual friends between any vertices.”**

NOTE: The graph contains no self-loops or parallel edges. Vertex labels are mapped by a mapper array (Type: String).

The function/method `maxMutual()` takes a graph `g` represented in a 2D Matrix, and the mapper as a 1D Array of Strings in the parameter.

Sample Input	Sample Output
<pre>mapper= ["Alice","Harry","Nimo","Nimas","Bob","Moana"] g = 0 1 1 0 0 0 1 0 1 1 0 0 1 1 0 1 0 0 0 1 1 0 1 0 0 0 0 1 0 1 0 0 0 0 1 0 </pre>	Alice and Nimas have 2 mutual friend(s). Explanation: Alice and Nimas have 2 mutual friends, Harry and Nimo, which is the maximum in this graph.

Function/Method Skeletons:

Python Notation	Java Notation
<pre>def maxMutual(g, mapper): # Write your code here</pre>	<pre>static void maxMutual(int [][] g, String[] mapper) { // Write your code here }</pre>